

操作系统 · 知识手册 11 张卡片 · 约 35 分

一、操作系统概述

高频 · 必考 / 选择 · 约 6 分钟

四个特征、两种态与系统调用

OS 的四个基本特征：并发（同一时间段内多个事件都发生了，宏观同时；并行才是微观同时，靠多核实现）、共享（互斥共享资源如打印机、虚拟共享如磁盘）、虚拟（时分复用把物理实体变多个逻辑副本：虚存、虚拟终端；空分复用如 SPOOLing 把一台打印机虚拟成多台）、异步（进程以不可预知的速度推进，但结果确定）。并发和共享是最基本的两个特征，互为存在条件。

内核态 / 用户态

- 内核态可执行特权指令（设置时钟、启动 I/O、修改 PSW、关中断、访管指令本身等），可访问全部地址空间；用户态执行特权指令会触发异常。
- 从用户态进入内核态只能靠中断/异常/陷入（trap），包括系统调用；从内核态返回用户态由 OS 主动切换。
- 系统调用是程序接口（API 的一种，如 read/write/fork），库函数（如 printf）在用户态实现，可能内部再调系统调用——「调用库函数一定陷入内核」是错的（puts 有缓冲，未刷新时不进入内核）。
- 一次系统调用要经历：参数传递→陷入指令→保存现场→查系统调用表执行→恢复现场返回，所以开销远大于普通函数调用（这也是「频繁系统调用是性能杀手」的原因）。

OS 的主要功能：处理器管理（进程/线程调度）、存储器管理、文件管理、设备管理，外加用户接口。目标：方便性、有效性、可扩充性、开放性。

一、操作系统结构

选择 · 约 6 分钟

大内核、微内核与虚拟机

- **大内核（宏内核）**：把进程管理、内存、文件系统、设备驱动都放内核态，直接函数调用，效率高但任一模块出错就整机崩溃，体积大。
- **微内核**：内核只保留最核心功能（少量 IPC、调度、内存原语），文件/设备/网络等服务放用户态服务器进程。优点是可扩展、可靠性高、易移植、易形式化验证；缺点是服务请求要多次进程切换与消息传递，性能较低（这是最常考的对比）。
- **分层结构**：由内到外逐层封装，调试容易，但跨层调用要经过多层，有性能损失。
- **模块化（可加载内核模块）**：Linux 的做法，兼具面向对象与动态链接，性能接近大内核。
- **客户-服务器模式**：微内核常见形态。
- **虚拟机**：Type-1 裸机直接管理硬件（Vmware ESXi、Xen），Type-2 寄宿在宿主 OS 之上（VirtualBox）；虚拟机监视器 VMM 负责虚拟 CPU/内存/设备，可分全虚拟化与半虚拟化、硬件辅助（VT-x）。
- **实时系统**：硬实时（期限绝对不可违反，如飞行控制）与软实时（偶尔超期可容忍）；关键指标是中断延迟与调度延迟的可预测性，不是平均吞吐量。

二、进程与线程

高频 · 必考 / 大题 · 约 6 分钟

状态转换、线程与 IPC

三态模型：就绪 ↔ 运行（调度/时间片到），运行 → 阻塞（等 I/O、等信号量，主动），阻塞 → 就绪（事件到达，被动）。注意：阻塞不能直接变运行；运行不能直接变就绪→阻塞。五态再加上创建态、终止态；挂起态用于对换（就绪挂起/阻塞挂起）。

- 进程 = 程序段 + 数据段 + PCB（进程控制块：PID、状态、优先级、程序计数器与寄存器现场、内存指针、打开文件表、资源清单）。PCB 是进程存在的唯一标志。
- 进程切换保存/恢复的是 PCB + 用户栈 + 页表基址等；线程切换比进程切换快，但仍是内核开销；同一进程的线程共享地址空间与文件描述符、独占栈与寄存器与 PC。
- 线程实现：用户级线程 ULT（内核不知，一个阻塞系统调用会阻塞整个进程，切换极快）、内核级 KLT（调度单位，切换有内核开销）、混合式。LWP 轻进程作为中间层。
- fork 创建子进程（返回 0 给子、返回子 PID 给父，父子共享代码段、各自拷贝数据段），exec 系列替换映像。

IPC 六大手段（必须会排优缺点）

方式	特点
管道 pipe	半双工、字节流、父子或有亲缘关系进程；容量有限
命名管道 FIFO	无亲缘关系也可，存在于文件系统
消息队列	有格式的报文链表，可随机读写、不受大小限制，需用户态到内核态拷贝
共享内存	最快（建立映射后不再经过内核拷贝），但需另配同步机制
信号量	主要用于同步与互斥，信息量小
信号 signal	异步通知，软件层的中断，可靠性弱
Socket	可用于不同主机间通信

二、处理机调度

高频 · 必考 / 大题 · 约 6 分钟

调度算法与三个指标

调度层次：高级（长程，决定谁进入内存）、中级（中程，对换出去/来，缓解内存压力）、低级（短程，从就绪队列挑一个上 CPU，最频繁）。

评价指标

- 周转时间 = 完成时间 - 到达时间；带权周转 = 周转 / 实际运行时间 (≥ 1 ，越接近 1 越好)。
- 响应时间：从提交到首次响应（交互系统关键）。
- 吞吐量、CPU 利用率、就绪队列长度。

算法（给定表算平均周转是必考大题）

- FCFS：简单、无饥饿，但对长作业有利、短作业吃亏，平均周转差。
- SJTF/SRTF（短作业优先/最短剩余时间）：平均周转最优，但长作业可能饥饿；非抢占 vs 抢占要分清。
- HRRN 高响应比优先：响应比 = (等待时间 + 要求服务时间) / 要求服务时间 = 1 + 等待/服务，兼顾长短作业，解决饥饿（等得越久优先级越高）。
- 优先级调度：静态/动态优先级；饥饿问题用老化 (aging) 解决。
- 时间片轮转 RR：响应好，时间片太大退化成 FCFS，太小切换开销占主导；通常取几十毫秒。
- 多级反馈队列：兼顾各指标，不必预知作业长度，是现实系统常用方案。

调度时机：running 进程主动放弃（终止、请求 I/O、调用 wait、主动让出）或时间片到 / 有更高优先级进程就绪（抢占式）。不能在进程处于内核临界区时随意切换（早期 Unix 只在返回用户态时调度）。

二、同步与互斥

高频·必考 / 大题·约 6 分钟

信号量、管程与三个经典问题

临界区四原则：空闲让进、忙则等待、有限等待（不饥饿）、让权等待（等不到就阻塞，不空转）。

结构：entry → 临界区 → exit → remainder；互斥实现要有进入区检查、退出区唤醒。

信号量 S（P 等待、V 释放，均为原子操作）

- 整型信号量不满足让权等待（会忙等/自旋）。
- 记录型信号量：S.value + 阻塞队列 S.L；P 操作 S.value--，若 <0 则 S.L enqueue 并 block；V 操作 S.value++，若 ≤0 则从队列唤醒一个。S 初值 = 资源数；S < 0 时 |S| 就是阻塞进程数。
- 用法：互斥初值 1（P 前 V 后成对）；同步/前驱关系用初值 0 的信号量，在 A 之后 V、B 之前 P。
- 注意 P、V 顺序错 → 死锁；对同一信号量连续两次 P → 自锁。

管程 (monitor)：把共享数据与操作封装，内部只有一个条件变量队列，自动互斥，解决信号量「使用灵活但易错」的问题；条件变量 wait/signal 用于等待某条件成立。

三个经典问题（背结构）

- 生产者-消费者：mutex=1, empty=n, full=0；生产：P(empty)→P(mutex)→放→V(mutex)→V(full)。顺序颠倒会死锁。
- 读者-写者：读者优先（用 count 与 rmutex，第一个读者 P(rw)、最后一个 V(rw)）；写者优先要再加 wsem/3 号信号量；读写公平用「先来服务」/带ticket方案。
- 哲学家进餐：最多同时让 4 人拿筷子（计数信号量）、或奇数号先拿左、偶数号先拿右、或取双筷用餐程一次性拿（三种解法任写一种即可）。

自旋锁适合多处理器且临界区极短（不让权、不切换）；互斥锁会让权。

二、死锁

高频·必考 / 大题·约 6 分钟

四个必要条件与银行家算法

死锁 vs 饥饿 vs 忙等：死锁是两个以上进程互相等待、不用外力谁也无法推进。

四个必要条件（同时具备才会死锁，破坏任一即预防）：互斥、请求并保持（持有并等待）、不可剥夺、循环等待。

- 预防：静态分配（一次性申请全部资源，破坏「请求并保持」）、允许剥夺（破坏不可剥夺）、按序分配（给资源编号，只能递增申请，破坏循环等待，最常用）。缺点：资源利用率低、编号困难。
- 避免：银行家算法——申请前假定分配，再检查系统是否仍处于安全状态（存在一个安全序列能让所有进程依次完成）。不安全 $\neq$ 死锁，只是不保证避免。

手算套路：Need = Max - Allocation；从「Need $\leq$ Work 的进程」开始找，成功则 Work += 该进程 Allocation。按题目给的顺序（如进程编号）取第一个可行的，安全序列可能不唯一。

- 检测：资源分配图化简（找能被满足的进程并释放其资源，反复）；若全部结点可消去 $\rightarrow$ 无死锁。同类资源只有一个实例时该法充分必要。
- 解除：剥夺资源、撤销进程（代价最小优先）、回滚。

▷ 常见陷阱题：「死锁的四个条件同时成立」是死锁的必要条件；只有两个进程竞争时也完全可能死锁；「系统处于不安全状态」不等于「已经死锁」。

三、内存管理

高频·必考 / 大题·约 6 分钟

连续分配、分页与分段

连续分配

- 单一连续、固定分区（内部碎片）、动态分区（外部碎片）。
- 动态分区算法：首次适应（地址升序找第一个够大的，最快、高地址端碎片多）、最佳适应（按容量升序，留下最多无法利用的小碎片）、最坏适应（挑最大的空闲块，减少小碎片但大碎片很快耗尽）。首次适应与最佳适应更常用。
- 解决外部碎片：紧凑/拼接（移动进程，开销大、需要动态重定位）。

分页：页（逻辑）与页框/物理块（物理）等长，内部碎片平均半页、无外部碎片；地址 = 页号 | 页内偏移（页大小 2^k 时低 k 位就是偏移）。

- 页表实现「页号→块号」映射，本质是硬件可加速的数组；访存要两次（查页表 + 取数据），所以配 TLB（快表）。
- 有效访问时间 $EAT = TLB\text{命中时间} + (1-\alpha)(\text{页表访存} + \text{数据访存}) + \text{数据访存}$ ，按题目给的结构算，别背一个万能公式。
- 大页表 → 多级页表（省连续内存、按需分配）或逆页表（哈希）；inverted page table 每个物理块一项，适合共享地址空间。
- 页面大小权衡：大页 → 页表小、TLB 命中高、但内部碎片与读入开销大。

分段：按逻辑单位（主程序、子程序、数组），段号+段内偏移，段长可变，需基址+限长寄存器的越界保护；优点是便于共享、保护、动态链接，缺点是外部碎片。段页式：段内分页，地址 = 段号 | 页号 | 偏移，需查 3 次表（配 TLB 减少）。

共享与保护：页/段的描述符里有访问权限位；硬链接是共享数据块的多个目录项。

三、虚拟内存

高频·必考 / 大题·约 6 分钟

请求分页、置换与工作集

基础：局部性原理 → 虚存「请求装入 + 置换」。请求分页在分页上加页表项的有效性位、修改位 (dirty)、访问位、外存地址、保护位，并支持缺页中断。

缺页处理流程：访存 → 页表无效 → 产生缺页中断 (trap, 属于内中断/异常) → OS 找空闲帧 (没有就置换) → 必要时把被换出页写回外存 → 修改页表与 TLB → 重新执行被中断的指令。缺页是「重执行型」中断，与其他中断不同。

置换算法

- OPT 最佳 (淘汰将来最久不用)：理论上最好、无法实现，作评价基准。
- FIFO：实现简单，可能出现 Belady 异常 (增加物理帧反而缺页更多)。
- LRU：用最近使用历史，性能接近 OPT，需要堆栈或计数器硬件，代价高。
- Clock/改进型 Clock：用访问位 (+修改位) 环形扫描，接近 LRU 而开销小，实际系统常用。
- 页面大小、分配帧数、局部/全局置换策略共同决定缺页率。

抖动 (thrashing)：刚换入又换出，页面频繁倒换而用户进程得不到服务，CPU 利用率暴跌。成因是多道程序度过高 / 分配帧数低于需求。防治：工作集模型 WSS(t) (某时刻前 Δ 时间窗口内访问的页面集合) —— 让进程在内存中至少驻留其工作集；工作集窗口 Δ 要足够大以覆盖局部性区间；用缺页率 PF 动态调整分配帧数 (PCP 进程控制策略)。

请求分段有内部碎片少但外部碎片、地址转换需要段表 + 长度；请求段页式兼具两者。

四、文件管理

高频 · 必考 / 大题 · 约 6 分钟

目录、物理结构与链接

文件控制块 FCB 存放文件名、物理地址、逻辑结构、物理结构、存取控制、使用信息、建立时间等——文件存在的标志。UNIX 把「名→索引节点号」分离出来形成目录项，索引节点 (inode) 存元数据，减少检索时的访存次数 (目录项只放名字和 i 号这一设计年年考)。

目录结构：单级 (不能重名)、两级 (隔离不同用户)、树形 (解决重名、检索路径长)、无环图 DAG (实现共享)。

逻辑结构：流式文件 (无结构，UNIX) 与记录式文件 (定长/变长记录，有顺序/随机/索引访问方式)。

物理 (外存分配) 结构

方式	优点	缺点
连续分配	顺序与随机访问都快 (支持直接访问)	外部碎片、文件不易增长
链接分配 (隐式)	无外部碎片、易扩展	只能顺序访问，指针占空间，一个坏块丢一串
链接分配 (显式 FAT)	检索快 (表在内存)	FAT 占内存，安全性差
索引分配 (一级/二级/混合)	支持直接访问、易增长	索引块开销；大文件用链式/多级/混合索引 (如 inode 的 12 直接 + 1 一级间接 + 1 二级间接 + 1 三级间接)

混合索引的最大文件长度计算题：直接块 + 间接各级块数 × 每块指针数 × 块大小，按题目给的块大小与指针长度分层算。

共享：硬链接 (增加索引节点引用计数，删除文件即删目录项，计数为 0 才真正释放；不能跨文件系统、多数系统不允许对目录硬链接)；软链接/符号链接 (存放路径名，可跨系统、可指目录，但访问时要多一次解析、目标删除会变成悬空链接)。

保护：存取控制列表 ACL / 权限位 (rwx 三位一组，分别属主/组/其他，用八进制表示如 754)。空闲空间管理：空闲表、空闲链表 (串成空闲盘块链)、位示图 (每位对应一块，1/0 表示占用；块号 ↔ 位号换算按每字 w 位、盘块号从 0 或 1 起算两种口径)，成组链接法 (UNIX，把空闲块号分组，每组首块记录下一组，避免整张表常驻内存)。文件系统挂载：将已建好的文件系统目录树接到当前目录树某结点；挂载后要先写超级块 (含文件系统的元信息)。

五、设备管理

高频·必考 / 大题·约 6 分钟

I/O 层次、缓冲与 SPOOLing

I/O 软件层次（自顶向下）：用户层软件（如 `printf`）→ 设备独立性软件（统一接口、缓冲、错误报告、分配与释放设备、SPOOLing）→ 设备驱动程序 → 中断处理程序 → 硬件（I/O 控制器与设备）。设备无关性：用户使用逻辑设备名，由系统映射到物理设备（查 LUT 逻辑设备表），使程序不依赖具体设备。

I/O 控制方式：程序查询 → 中断驱动 → DMA（块为单位、需总线权、CPU 只在开始结束介入）→ 通道（通道程序、独立管理 I/O）→ I/O 处理机。中断方式每次传输一个/几个字都要 CPU 参与；DMA 一个数据块一次中断。

缓冲区的作用：缓解 CPU 与设备速度不匹配、减少中断次数、提高并行性、解决数据 size 不匹配。

- 单缓冲：一块数据从设备送到缓冲区再送到工作区，时间 = $\max(C, T) + M$ 。
- 双缓冲：设备输入与用户处理可并行，时间 $\approx \max(C+T, M)$ （按题给参数推）。
- 循环缓冲区用于连续数据流；缓冲技术是软件实现，SPOOLing 是「用磁盘当缓冲池把独占设备改造成共享设备」。

SPOOLing（假脱机）：输入井/输出井（磁盘）+ 输入/输出缓冲区（内存）+ 预输入/缓输出程序。典型应用是共享打印机：各进程把输出写到自己的输出井文件中，守护进程排队打印。优点：把独占设备虚拟为多个逻辑设备，提高 CPU 与设备并行度；注意 SPOOLing 依赖磁盘且需要后台进程配合。

设备分配数据结构：系统设备表 SDT、设备控制表 DCT、控制器表 COCT、通道表 CHNT；分配策略：静态（安全、利用率低）与动态（需配合死锁避免）。

设备独立性的实现：逻辑设备名 → 物理设备名，靠 LUT + 驱动程序入口，更换设备不必改程序。

磁盘调度与 SSD

磁盘地址：柱面（磁道）号 | 磁头号 | 扇区号。一次访问的时间 = 寻道时间 + 旋转延迟（平均=转一圈时间/2）+ 传输时间。

调度算法（必会算总寻道道数）

- FCFS：按请求顺序，公平，但磁头来回幅度大。
- SSTF：最近的先服务，性能好但可能饥饿（边缘请求长期得不到服务）。
- SCAN 电梯算法：沿一个方向走到端点再折返 —— 会扫描到最外侧柱面；LOOK 提前在该方向最后一个请求处折返。
- C-SCAN / C-LOOK：单向服务，到头后快速返回起点，等待时间更均匀（对多个请求公平，适合高负载）。

SSD 固态硬盘：闪存（NAND，按页读、按块擦写）、没有寻道与旋转延迟、随机读极快；写入前必须先擦除整块，因此有磨损均衡（延长寿命）与垃圾回收；用 FTL 做逻辑→物理页映射并支持就地更新（写新位置+改映射+回收旧块）。RAID：0 条带（无冗余、快）、1 镜像（容量减半、可并发读）、5 分布式奇偶（n 块盘损失 1 块容量、允许坏 1 块、写惩罚）、6 双校验、10 镜像+条带。

▷ 计算题里注意：题目给的是「磁头当前所在柱面 + 上一移动方向」才能判断 SCAN/LOOK；求「总移动道数」时把折返段也算进去，别忘了返回起点的这一段（C-SCAN 要算回程，LOOK 只回到最后一个请求）。

与组成原理重叠多（存储、I/O），概念辨析题多，背结构就能拿分。

「考频」是按题型惯例给出的人工判断，不是官方统计；大纲与分值以当年《考试大纲》《考试分析》为准。

来源：本地手写（依据 408 考试大纲与通行教材整理，不复制任何第三方受版权保护的讲义）；构建于 2026-09-28 10:15。公式渲染失败的条目会以红色等宽字体保留原文。